

برای تشخیص ارقام از دیتاست‌های صوتی، مدل طراحی شده در این پروژه شامل بلوک‌هایی از جمله Attention، Positional Encoding و ساختار Transformer است. مدل طراحی شده فاقد هرگونه بلوک RNN بوده و از یک معماری مدرن‌تر برای پردازش داده‌های توالی‌محور استفاده می‌کند.

اجزای مدل طراحی شده

مدل پیشنهادی شامل اجزای زیر است:

۱. دیتاست سفارشی:

کلاس AudioDataset برای مدیریت و آماده‌سازی داده‌ها طراحی شده است. این کلاس فایل‌های صوتی را خوانده و به شکل‌دهی آن‌ها به طول مشخص (با استفاده از پدینگ یا برش) می‌پردازد. هر فایل صوتی، لیبل از نام فایل استخراج می‌شود (اولین رقم نام فایل).

```
# Custom Dataset for Audio Data
import torch.nn.functional as F

class AudioDataset(Dataset):
    def __init__(self, folder_path, transform=None, max_len=18000): # max_len defines the desired sequence length
        self.file_paths = []
        self.labels = []
        self.transform = transform
        self.max_len = max_len

        for file_name in os.listdir(folder_path):
            file_path = os.path.join(folder_path, file_name)
            if os.path.isfile(file_path) and file_name[0].isdigit():
                self.file_paths.append(file_path)
                self.labels.append(int(file_name[0])) # Extract label from the first character of the file name

    def __len__(self):
        return len(self.file_paths)

    def __getitem__(self, idx):
        file_path = self.file_paths[idx]
        label = self.labels[idx]

        waveform, sample_rate = torchaudio.load(file_path)

        # Padding or Trimming the waveform
        if waveform.size(1) > self.max_len:
            waveform = waveform[:, :self.max_len]
        else:
            waveform = F.pad(waveform, (0, self.max_len - waveform.size(1))) # Pad with zeros

        if self.transform:
            features = self.transform(waveform)
        else:
            features = waveform

        return features, label
```

۲. ویژگی‌های صوتی:

از ویژگی‌های MFCC (Mel Frequency Cepstral Coefficients) برای نمایش داده‌های صوتی استفاده شده است.

استخراج ویژگی‌ها با استفاده از تابع `extract_mfcc` انجام می‌شود.

```
# Feature Extraction (MFCC)
def extract_mfcc(sample_rate, n_mfcc=40):
    return torchaudio.transforms.MFCC(
        sample_rate=sample_rate,
        n_mfcc=n_mfcc,
        melkwargs={"n_fft": 400, "hop_length": 160, "n_mels": 40}
    )
```

ویژگی MFCC چیست؟

MFCC یا **Mel-Frequency Cepstral Coefficients** یکی از رایج‌ترین ویژگی‌ها برای پردازش سیگنال‌های صوتی است که به‌طور گسترده در پردازش گفتار، تشخیص گفتار و سایر برنامه‌های صوتی استفاده می‌شود.

MFCC ویژگی‌هایی از سیگنال صوتی را استخراج می‌کند که نزدیک به نحوه درک صدا توسط گوش انسان هستند. این ویژگی‌ها نمایشی فشرده از اطلاعات طیفی صدا را ارائه می‌دهند و بر اساس مقیاس فرکانس مل (**Mel Scale**) ساخته می‌شوند.

MFCC اطلاعات اصلی سیگنال صوتی (مانند شکل طیفی) را به صورت فشرده استخراج می‌کند و به ما اجازه می‌دهد که خصوصیات مهم گفتاری (مانند تن صدا و فرکانس‌های اصلی) را به جای کل سیگنال، تحلیل کنیم. این ویژگی‌ها برای تشخیص الگوهای صوتی مانند کلمات، صداها و تلفظ‌ها بسیار مفید هستند.

مراحل محاسبه MFCC

MFCC در چند مرحله از یک سیگنال صوتی محاسبه می‌شود:

قطعه‌بندی سیگنال صوتی (Framing):

سیگنال صوتی پیوسته به قطعات کوچکتر (معمولاً به طول ۲۰ تا ۴۰ میلی ثانیه) تقسیم می‌شود. این قطعات به نام **فریم** (frame) شناخته می‌شوند.

اعمال پنجره‌گذاری (Windowing):

برای جلوگیری از نشت فرکانس‌ها، یک پنجره (معمولاً پنجره هامینگ) روی هر فریم اعمال می‌شود.

محاسبه FFT (Fast Fourier Transform):

برای هر فریم، طیف فرکانسی با استفاده از FFT محاسبه می‌شود. این مرحله سیگنال را از حوزه زمان به حوزه فرکانس می‌برد.

اعمال مقیاس مل (Mel Scale) :

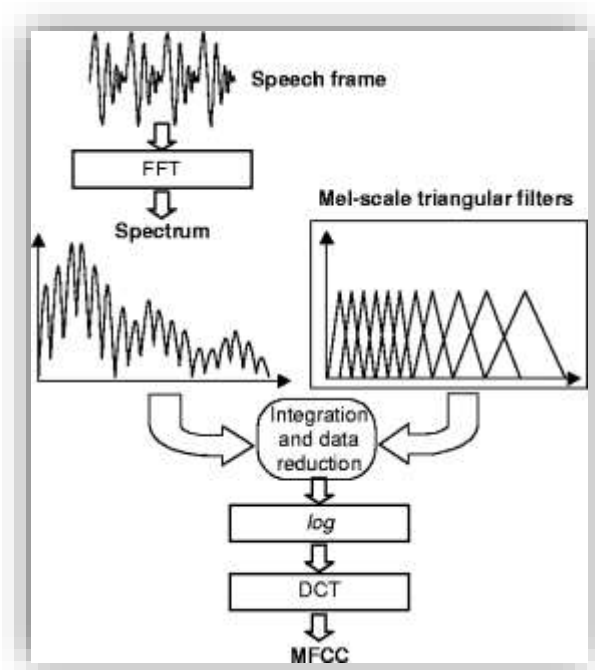
طیف فرکانسی به مقیاس مل نگاشت می‌شود. مقیاس مل شبیه نحوه درک گوش انسان از فرکانس‌ها است. این مقیاس در فرکانس‌های پایین جزئیات بیشتری دارد و در فرکانس‌های بالا جزئیات کمتری.

محاسبه توان انرژی (Log Energy) :

توان انرژی در هر باند مل محاسبه می‌شود و سپس لگاریتم گرفته می‌شود. این مرحله به دلیل نحوه درک غیرخطی گوش انسان از انرژی ضروری است.

تبدیل کسینوسی گسسته (DCT) :

DCT روی مقادیر به‌دست‌آمده اعمال می‌شود تا ضرایب MFCC استخراج شوند. این ضرایب اساساً اطلاعات فشرده طیفی سیگنال هستند و به ما اجازه می‌دهند که مهم‌ترین ویژگی‌ها را استخراج کنیم.



۳. معماری مدل:

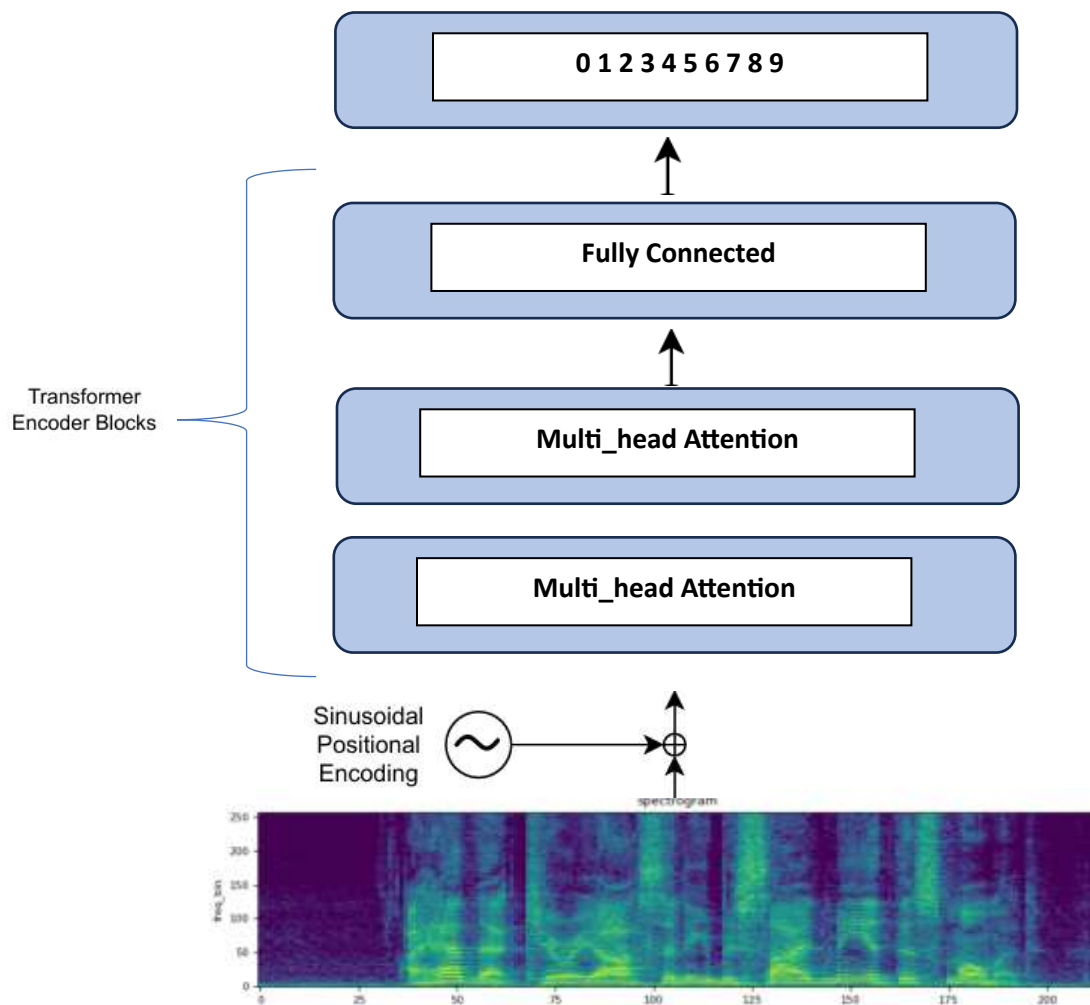
مدل طراحی شده با استفاده از ساختار Transformer شامل اجزای زیر است :

لایه تعبیه‌سازی : داده‌های ورودی را به فضای بُعد بالاتر (embedding) نگاشت می‌کند.

رمزگذاری موقعیتی (Positional Encoding): اطلاعات ترتیب زمانی به داده‌ها اضافه می‌شود تا مدل بتواند توالی را درک کند.

بلوک Transformer Encoder: شامل چندین لایه Attention چند سری (Multi-head Attention) و لایه‌های کاملاً متصل است.

لایه نهایی (Fully Connected Layer): برای نگاشت ویژگی‌ها به خروجی‌ها (احتمالات کلاس‌ها).



در این معماری ما فقط از انکودر استفاده کردیم زیرا هدف ما طبقه‌بندی است، نه تولید توالی (مانند ترجمه زبان). یک **انکودر ترانسفورمر** برای استخراج ویژگی‌های معنایی از داده‌های صوتی استفاده می‌کنیم و این ویژگی‌ها به یک لایه کاملاً متصل (Fully Connected) داده می‌شوند تا خروجی طبقه‌بندی شود.

۴. هایپرپارامترها:

تعداد لایه‌های Transformer Encoder : ۲

تعداد سرهای Attention : ۴

بعد مدل: ۱۲۸

تعداد کلاس‌ها: ۱۰ (برای ارقام ۰ تا ۹)

طول ورودی صوتی: ۱۶۰۰۰ نمونه (۱ ثانیه با نرخ نمونه‌برداری ۱۶ کیلوهرتز).

۵. فرآیند آموزش و ارزیابی

آماده‌سازی داده‌ها:

دیتاست به سه بخش تقسیم شده است :

80% برای آموزش

10% برای اعتبارسنجی

10% برای آزمایش

از DataLoader برای بارگذاری دسته‌ای داده‌ها استفاده شده است.

۶. آموزش مدل:

تابع هزینه (Loss Function) : CrossEntropyLoss

بهینه‌ساز (Optimizer) : Adam با نرخ یادگیری ۰.۰۰۱

مدل با تعداد ۵۰ اپیک آموزش داده شده است که خلاصه‌ای از نتایج آن به صورت زیر است :

```
Epoch [40/50], Loss: 0.0071, Train Accuracy: 100.00%  
Validation Loss: 0.0299, Validation Accuracy: 99.00%  
Epoch [41/50], Loss: 0.0063, Train Accuracy: 100.00%  
Validation Loss: 0.0339, Validation Accuracy: 99.00%  
Epoch [42/50], Loss: 0.0058, Train Accuracy: 100.00%  
Validation Loss: 0.0310, Validation Accuracy: 99.33%  
Epoch [43/50], Loss: 0.0056, Train Accuracy: 100.00%  
Validation Loss: 0.0228, Validation Accuracy: 99.33%  
Epoch [44/50], Loss: 0.0054, Train Accuracy: 100.00%  
Validation Loss: 0.0242, Validation Accuracy: 99.33%  
Epoch [45/50], Loss: 0.0059, Train Accuracy: 100.00%  
Validation Loss: 0.0246, Validation Accuracy: 99.33%  
Epoch [46/50], Loss: 0.0048, Train Accuracy: 100.00%  
Validation Loss: 0.0248, Validation Accuracy: 99.33%  
Epoch [47/50], Loss: 0.0045, Train Accuracy: 100.00%  
Validation Loss: 0.0215, Validation Accuracy: 99.33%  
Epoch [48/50], Loss: 0.0044, Train Accuracy: 100.00%  
Validation Loss: 0.0244, Validation Accuracy: 99.00%  
Epoch [49/50], Loss: 0.0042, Train Accuracy: 100.00%  
Validation Loss: 0.0279, Validation Accuracy: 99.33%  
Epoch [50/50], Loss: 0.0038, Train Accuracy: 100.00%  
Validation Loss: 0.0282, Validation Accuracy: 99.00%
```

۷. اعتبارسنجی و ارزیابی:

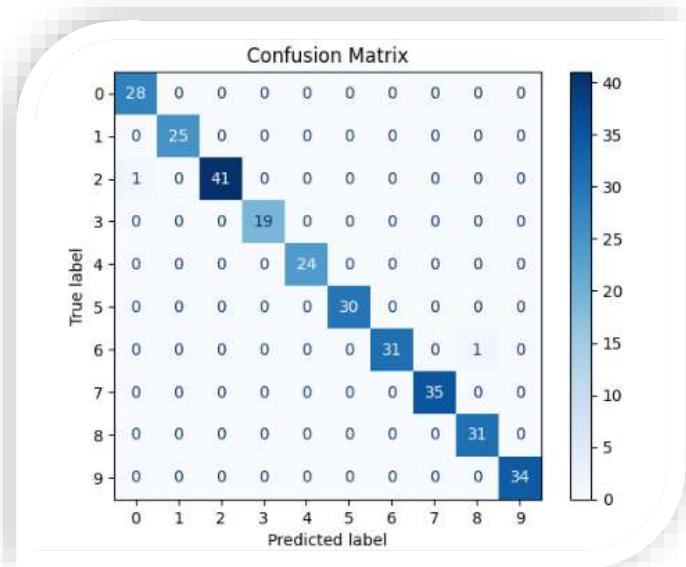
همانطور که در قسمت قبل مشخص است، دقت مدل در طول هر دوره برای مجموعه اعتبارسنجی محاسبه شده است.

در پایان آموزش، مدل روی مجموعه تست ارزیابی شده است که نتیجه ی آن به صورت زیر است:

Test Accuracy: 99.33%

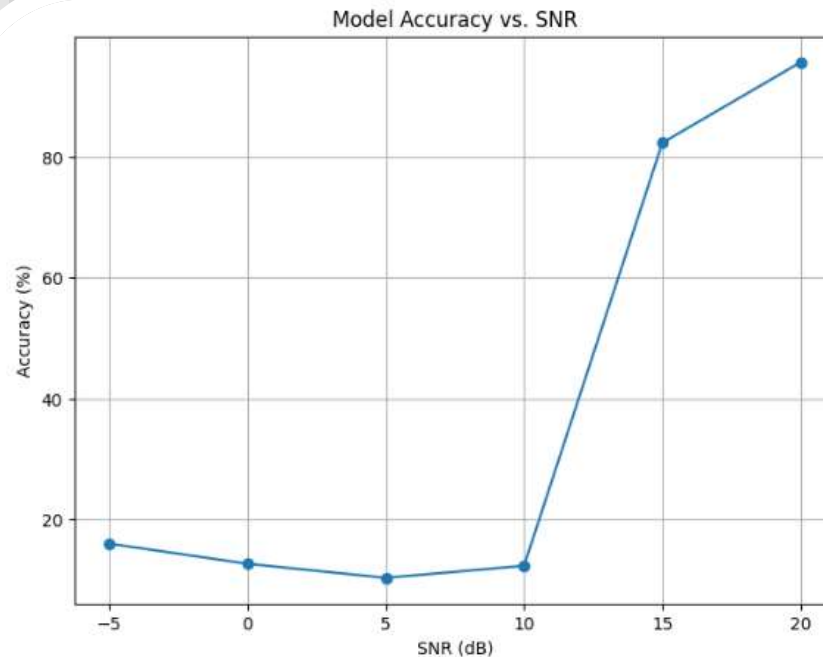
۸. ماتریس درهم ریختگی (Confusion Matrix):

ماتریس درهم ریختگی این مدل روی داده های تست به شکل زیر است :



در قسمت بعد مقداری نویز سفید به مدل اضافه کردیم و عملکرد آن را در برابر این نویز ها بررسی کردیم که نتیجه ی این بررسی ها به صورت زیر است :

SNR: 20 dB, Accuracy: 95.67%
SNR: 15 dB, Accuracy: 82.33%
SNR: 10 dB, Accuracy: 12.33%
SNR: 5 dB, Accuracy: 10.33%
SNR: 0 dB, Accuracy: 12.67%
SNR: -5 dB, Accuracy: 16.00%



همانطور که در نمودار بالا قابل مشاهده است ، محور افقی نشان دهنده ی مقادیر **SNR** است که معمولاً از مقادیر مثبت (مثلاً ۲۰ دسی بل) شروع شده و به مقادیر منفی (مثلاً -۵ دسی بل) ختم می شود. هرچه **SNR** بیشتر باشد، نسبت قدرت سیگنال به نویز بیشتر است، و داده ها تمیزتر هستند.

محور عمودی بیانگر دقت (Accuracy) مدل است که در **SNR** بالا، دقت معمولاً نزدیک به حداکثر مقدار ممکن است و در **SNR** پایین (نویز بالا) ، دقت به تدریج کاهش می یابد.

در قسمت بعد برای مقاوم سازی مدل در برابر نویز دو راهکار را امتحان کردیم که اولین راهکار استفاده از Regularization و Robust Loss Functions بود.

استفاده از Regularization :

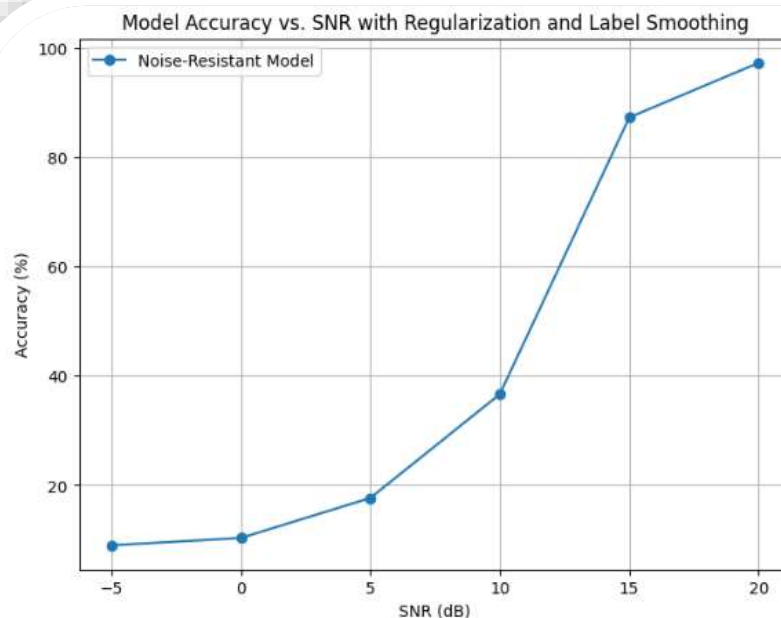
اضافه کردن لایه‌های Dropout یا Weight Decay به مدل برای افزایش توانایی تعمیم مدل.

استفاده از Robust Loss Functions :

استفاده از توابع هزینه‌ای مانند Label Smoothing که نویز را در نظر می‌گیرند.

این تغییرات را به مدل اعمال کردیم که نتیجه‌ی مطلوبی حاصل نشد :

SNR: 20 dB, Accuracy: 97.33%
SNR: 15 dB, Accuracy: 87.33%
SNR: 10 dB, Accuracy: 36.67%
SNR: 5 dB, Accuracy: 17.67%
SNR: 0 dB, Accuracy: 10.33%
SNR: -5 dB, Accuracy: 9.00%



به عنوان راهکار دوم استفاده از تکنیک داده‌افزایی (Data Augmentation) را امتحان کردیم .

روش پیشنهادی:

افزودن نویز به داده‌های آموزشی (Data Augmentation) :

به داده‌های آموزشی نویز سفید با SNR های مختلف اضافه کردیم تا مدل در برابر نویز مقاوم شود. این کار باعث می‌شود که مدل طی فرآیند آموزش، داده‌های نویزی را نیز مشاهده کند و عملکرد بهتری روی داده‌های نویزی داشته باشد.

تغییر ساختار مدل:

Dropout یا Normalization را به مدل اضافه کردیم که می‌تواند عملکرد مدل را بهبود دهد.

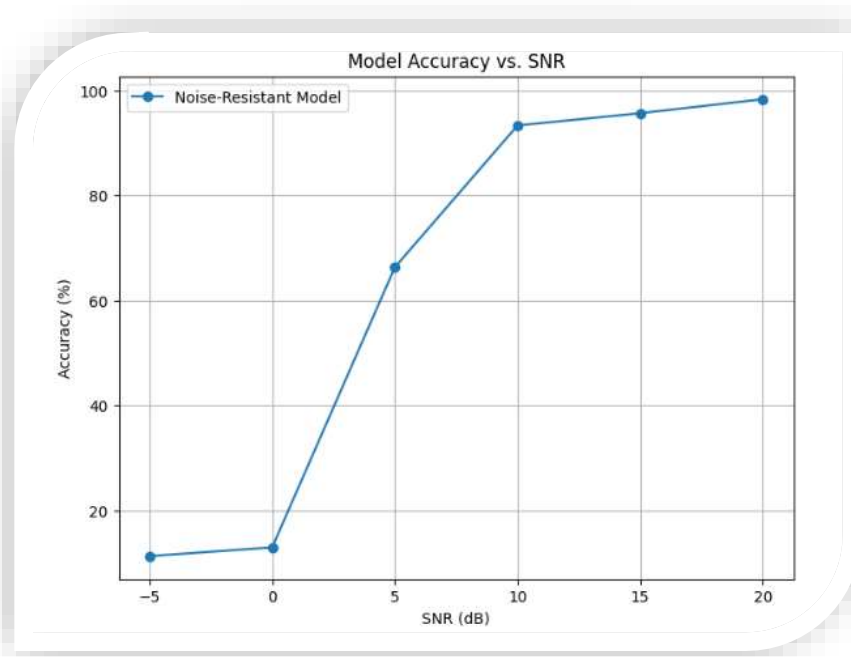
از معماری‌های مقاوم به نویز استفاده کردیم ، مثل ترکیب ویژگی‌های استخراج‌شده از سیگنال اصلی و نویز.

تغییر تابع هزینه (Loss Function) :

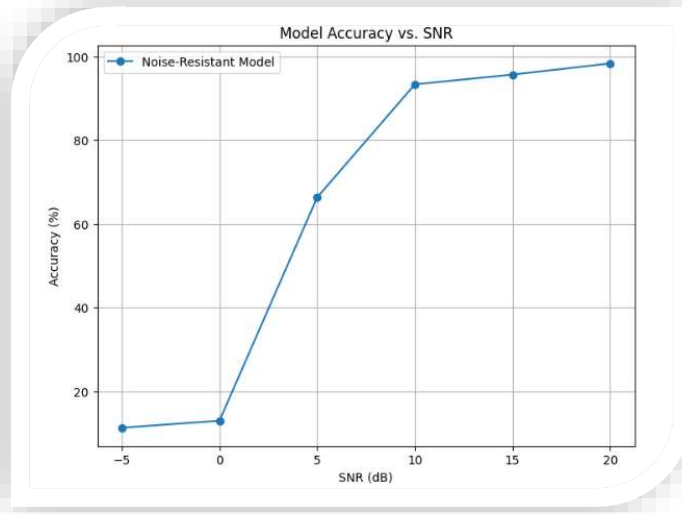
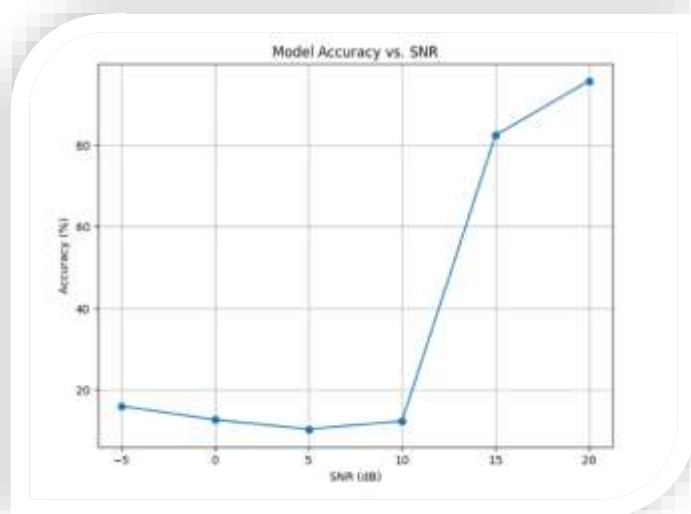
از توابع هزینه‌ای که نویز را در نظر می‌گیرند استفاده کردیم ، مثل CTC Loss و اضافه کردن یک وزن به داده‌های نویزی.

این تغییرات باعث بهبود عملکرد مدل به صورت چشمگیری شد که در زیر می‌توانید آن را مشاهده کنید :

SNR: 20 dB, Accuracy: 98.33%
SNR: 15 dB, Accuracy: 95.67%
SNR: 10 dB, Accuracy: 93.33%
SNR: 5 dB, Accuracy: 66.33%
SNR: 0 dB, Accuracy: 13.00%
SNR: -5 dB, Accuracy: 11.33%



به عنوان مقایسه ، نمودار مدل اولیه (سمت چپ) را در مقابل نمودار مدل بهبود یافته با روش Data Augmentation (سمت راست) مشاهده می کنید :



که به وضوح بهبود دقت آن قابل مشاهده است .